

**Somaiya Vidyavihar University**  
**K. J. Somaiya School of Engineering, Mumbai-77**

**Batch: C-3      Roll No.: 16010123325**

**Experiment / Assignment / Tutorial No. 3**

**Grade: AA / AB / BB / BC / CC / CD / DD**

**Signature of the Staff In-charge with date**

**Title: Demonstration of Buffer Overflow, Integer Overflow, Null Terminated String, Race Condition**

**Objective:** Implementation of the program

- Buffer Overflow.
- Buffer Overwrite.

Exploring other forms / types of Buffer Overflow Attack

- Integer Overflow
- Null Termination of String
- Incomplete mediation
- Race Conditions.

**Expected Outcome of Experiment:**

|                       |                                                                             |
|-----------------------|-----------------------------------------------------------------------------|
| <b>Course Outcome</b> | <b>After successful completion of the course students should be able to</b> |
| CO2                   | Apply various cryptographic algorithms for software security.               |

**Books/ Journals/ Websites referred:**

- William Stallings, Cryptography and Network Security: Principles and Practice, Pearson.
- Charles P. Pfleeger and Shari Lawrence Pfleeger, Security in Computing, Prentice Hall.
- OWASP Foundation – Buffer Overflow Attack Documentation.

**Pre Lab/ Prior Concepts:**

- Basics of C/C++ programming
- Memory layout of a process (Stack, Heap, Data, Text segment)
- Arrays and pointers
- Function calls and stack frames
- Basic operating system concepts
- Introduction to software vulnerabilities and exploits

## **Somaiya Vidyavihar University**

### **K. J. Somaiya School of Engineering, Mumbai-77**

**Abstract:**

Buffer overflow is one of the most critical and commonly exploited software vulnerabilities caused by improper memory handling. This experiment demonstrates various types of vulnerabilities including buffer overflow, buffer overwrite, integer overflow, null-terminated string errors, incomplete mediation, and race conditions. Through practical implementation, students understand how insufficient input validation and improper boundary checking can lead to security breaches. The experiment also emphasizes the importance of secure coding practices and validation mechanisms to prevent such attacks and enhance software reliability.

**Related Theory:**

A buffer is a temporary memory storage area used to hold data while it is being transferred or processed. When a program fails to validate input size properly, it may allow data to exceed the allocated memory space, causing a buffer overflow.

**1. Buffer Overflow**

Occurs when more data is written to a buffer than it can hold, overwriting adjacent memory. This may lead to crashes, unexpected behavior, or execution of malicious code.

**2. Buffer Overwrite**

A form of overflow where adjacent variables or control data (such as return addresses) are overwritten due to improper bounds checking.

**3. Integer Overflow**

Occurs when an arithmetic operation exceeds the maximum value that a variable type can store. This can cause incorrect memory allocation sizes, leading to further vulnerabilities.

**4. Null Terminated String Error**

In C-based languages, strings are terminated by a null character (`\0`). If proper termination is missing, string-handling functions may read beyond intended memory, leading to overflow or data leakage.

**5. Race Condition**

Occurs when two or more processes access shared data simultaneously and the final result depends on the timing of execution. Attackers can exploit timing gaps to manipulate system behavior.

## Somaia Vidyavihar University K. J. Somaia School of Engineering, Mumbai-77

### Implementation Details:

#### 1. Buffer Overflow

```
#include <stdio.h>

int main() {
    char buffer[8];

    for(int i = 0; i < 100; i++) {
        buffer[i] = 'A';
    }

    printf("Completed\n");

    return 0;
}
```

#### Output-

```
shreya@Shreyas-MacBook-Air-2 exp3 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/" && gcc BufferOverflow.c -o BufferOverflow && "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/"BufferOverflow
Completed
zsh: abort      "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/"BufferOverflow
shreya@Shreyas-MacBook-Air-2 exp3 %
```

#### 2. Integer Overflow

```
#include <stdio.h>
#include <limits.h>

int main() {
    int max = INT_MAX;

    printf("Maximum signed int: %d\n", max);

    max = max + 1;

    printf("After overflow: %d\n", max);

    return 0;
}
```

#### Output-

```
shreya@Shreyas-MacBook-Air-2 exp3 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/" && gcc tempCodeRunnerFile.c -o tempCodeRunnerFile && "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/"tempCodeRunnerFile
Maximum signed int: 2147483647
After overflow: -2147483648
shreya@Shreyas-MacBook-Air-2 exp3 %
```

## Somaiya Vidyavihar University

### K. J. Somaiya School of Engineering, Mumbai-77

#### 3. Null Terminated String Error

```
#include <stdio.h>
#include <limits.h>

int main() {
    int max = INT_MAX;

    printf("Maximum signed int: %d\n", max);

    max = max + 1;

    printf("After overflow: %d\n", max);

    return 0;
}
```

#### Output-

```
cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/" && gcc NullTermination.c -o NullTermination
&& "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/"NullTermination
shreya@Shreyas-MacBook-Air-2 NIS % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/" && gcc NullTermination.c -o NullTermination && "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/"NullTermination
This is a sa
shreya@Shreyas-MacBook-Air-2 exp3 %
```

#### 4. Race Condition

```
#include <stdio.h>
#include <pthread.h>

int counter = 0; // shared variable

void* increment(void* arg) {
    for(int i = 0; i < 100000; i++) {
        counter++; // not thread-safe
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}
```

**Somaiya Vidyavihar University**  
**K. J. Somaiya School of Engineering, Mumbai-77**

```
printf("Expected counter value: 200000\n");  
printf("Final counter value: %d\n", counter);  
  
return 0;  
}
```

### Output-

```
● shreya@Shreyas-MacBook-Air-2 exp3 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/" && gcc RaceCondition.c -o RaceCondition && "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/"RaceCondition  
Expected counter value: 200000  
Final counter value: 106938  
○ shreya@Shreyas-MacBook-Air-2 exp3 %
```

### Conclusion:

This experiment provided practical understanding of various buffer-related vulnerabilities and demonstrated how improper input validation and memory management can compromise system security. By studying buffer overflow, integer overflow, null termination issues, incomplete mediation, and race conditions, students gained insight into common software weaknesses. The experiment emphasized the importance of proper validation, secure coding practices, and defensive programming techniques to prevent exploitation and ensure secure software development.

**Somaiya Vidyavihar University**  
**K. J. Somaiya School of Engineering, Mumbai-77**

**Post Lab Questions**

1. Can a format string vulnerability can lead to memory overwrite without directly using strcpy/gets? Justify your answer with suitable example.

Yes, A format string vulnerability occurs when user input is used directly as the format argument in functions like printf().

The %n format specifier in printf() writes the number of printed characters to a memory address, which allows an attacker to overwrite memory without using functions like strcpy() or gets().

```
#include <stdio.h>

int main() {
    int x = 0;
    printf("Before: %d\n", x);

    printf("AAAA%n", &x);    // %n writes number of characters printed

    printf("After: %d\n", x);
    return 0;
}
```

**Output-****Before: 0****After: 4**

2. Why should programmers guard against unsigned integer overflow even though the C standard defines its behavior?

In C, unsigned integer overflow is well-defined and wraps around using modular arithmetic.

For example, if the maximum value is exceeded, the value wraps back to 0.

Example:

```
#include <stdio.h>

int main() {
    unsigned int x = 4294967295; // max for 32-bit unsigned int
    x = x + 1;
    printf("%u\n", x);
}
```

Output: 0

## Somaiya Vidyavihar University

### K. J. Somaiya School of Engineering, Mumbai-77

Why it is dangerous:

Even though it is defined, unsigned overflow can still cause **serious bugs and security vulnerabilities**:

1. **Buffer size miscalculation**  
Overflow may result in allocating less memory than required.
2. **Security vulnerabilities**  
Attackers can exploit overflow to bypass checks or cause buffer overflows.
3. **Logic errors**  
Programs may behave incorrectly if calculations wrap unexpectedly.

3. Write a C code snippet demonstrating the TOCTOU vulnerability.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>

int main() {
    const char *filename = "example.txt";

    // Time-of-Check: Verify file exists and is writable
    struct stat st;
    if (stat(filename, &st) == 0 && (st.st_mode & S_IWUSR)) {
        printf("File is writable. Proceeding...\n");

        // Time-of-Use: Attempt to open and write (vulnerable window)
        int fd = open(filename, O_WRONLY);
        if (fd != -1) {
            write(fd, "Data", 4);
            close(fd);
            printf("Write successful.\n");
        } else {
            printf("Failed to open file.\n");
        }
    } else {
        printf("File not writable or does not exist.\n");
    }

    return 0;
}
```

**Output-**

**Somaiya Vidyavihar University**  
**K. J. Somaiya School of Engineering, Mumbai-77**

```
Lab > exp3 > example.txt
1 Hello.

shreya@Shreyas-MacBook-Air-2 exp3 % cd "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/" && gcc t
octou.c -o toctou && "/Users/shreya/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/"toctou
File is writable. Proceeding...
Write successful.
shreya@Shreyas-MacBook-Air-2 exp3 %
```

```
Lab ~/Desktop/CE/TY/Semester VI/NIS/Lab/exp3/NullTermination.c
1 Datao.
```